KNOWX: A Hybrid Retrieval-Augmented Generation Framework for Enterprise Knowledge Management

B. Manaswini, G. Babitha, G. Rohan, and P. Adithya Ram
Department of Artificial Intelligence and Data Science
K L (Deemed to be) University, India

***Abstract*—Enterprise organizations maintain extensive document collections containing financial, operational, and technical information. Locating relevant information in these collections can require users to search through lengthy reports and may be difficult when a query uses wording that differs from the source document. This paper presents KNOWX, a hybrid retrieval-augmented generation framework for question answering over enterprise documents. The system combines semantic retrieval using the all-MiniLM-L6-v2 sentence-embedding model and FAISS with keyword retrieval using BM25. Reciprocal Rank Fusion combines the two ranked lists, and a Cross-Encoder reranks the resulting candidates before context construction. A quantized Llama 3 8B Instruct model, executed locally through llama-cpp-python, generates a response from the retrieved context. The knowledge base contains nine documents collected from Microsoft, NVIDIA, and Infosys, yielding 6,256 text chunks. A preliminary automated known-source retrieval benchmark used 30 automatically generated questions and compared FAISS, BM25, Hybrid RRF, and Cross-Encoder reranking. In this benchmark, the Cross-Encoder achieved an MRR of 0.9833 and an nDCG@5 of 0.9877. These results are limited to the benchmark's single known relevant source chunk per question and should not be interpreted as independently validated answer accuracy. The paper describes the system architecture, implementation, preliminary retrieval results, and evaluation limitations.**

***Index Terms*—Retrieval-Augmented Generation, Enterprise Knowledge Management, Hybrid Information Retrieval, Semantic Search, BM25, FAISS, Reciprocal Rank Fusion, Cross-Encoder, Large Language Models.**

I. Introduction

Organizations store important knowledge in reports, annual filings, presentations, and other enterprise documents. Although these documents contain information needed for analysis and decision-making, users may have to inspect lengthy files manually to locate a specific fact. Keyword search can be useful when query terms overlap with document terms, but it may not retrieve a passage when the same idea is expressed using different words. Semantic retrieval addresses this wording problem by representing text as vectors, while keyword retrieval preserves exact-term matching.

Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. Instead of relying only on information encoded in a language model's parameters, a RAG system retrieves external passages and supplies them as context to a generator [1]. This design is relevant to enterprise question answering because answers can be conditioned on a selected document collection and accompanied by source information.

This paper presents KNOWX, a hybrid RAG framework developed to answer natural-language questions over documents from Microsoft, NVIDIA, and Infosys. KNOWX combines semantic search using FAISS and sentence embeddings with BM25 keyword retrieval. The two ranked lists are combined using Reciprocal Rank Fusion (RRF), followed by Cross-Encoder reranking. A locally executed, quantized Llama 3 8B Instruct model generates an answer from the selected context.

The objectives are to:

- build a document-processing pipeline for enterprise documents;
- index text chunks for semantic and keyword-based retrieval;
- combine complementary retrieval results using rank fusion;
- rerank retrieved passages before context construction;
- integrate local language-model inference with a web-based interface; and
- conduct a preliminary comparison of retrieval components.

The contributions are an implemented pipeline combining FAISS, BM25, RRF, and Cross-Encoder reranking; integration of quantized Llama 3 8B Instruct using local CPU inference; a corpus of nine documents and 6,256 chunks; and a preliminary known-source retrieval benchmark with its limitations stated explicitly.

II. Related Work

A. Retrieval-Augmented Generation

Lewis *et al.* introduced a framework combining a neural retriever with a sequence-to-sequence generator for knowledge-intensive tasks [1]. Their work motivates systems that retrieve external evidence at answer time rather than relying exclusively on a generator's parameters. KNOWX follows this general retrieve-then-generate pattern, using enterprise documents as its retrieval corpus.

B. Keyword and Semantic Retrieval

BM25 is a probabilistic ranking function that scores documents according to query-term occurrence and term statistics

[2]. Sentence-BERT uses siamese and triplet network structures to produce sentence embeddings suitable for similarity comparison [3]. These methods support complementary retrieval signals: BM25 emphasizes lexical overlap, while embedding-based retrieval can capture semantic similarity. KNOWX uses all-MiniLM-L6-v2 to create 384-dimensional sentence embeddings [4].

C. Rank Fusion and Reranking

Reciprocal Rank Fusion combines multiple ranked lists by accumulating reciprocal-rank contributions [5]. It provides a rank-based way to combine retrieval systems whose raw scores may not be directly comparable. Cross-Encoder reranking is a second-stage approach in which a query and candidate passage are jointly scored. Prior work has studied BERT-based passage reranking [6], while Sentence Transformers documentation describes a retrieve-and-rerank architecture [7]. KNOWX uses this two-stage design after initial retrieval.

D. Vector Search and Local Language Models

FAISS is a library for efficient similarity search and clustering of dense vectors [8], [9]. Meta introduced the Llama 3 family of language models in 2024 [10]. KNOWX uses a quantized GGUF model with llama-cpp-python for local inference [11]. These sources describe the underlying methods and tools; they do not establish the measured performance of KNOWX.

III. Methodology

A. Document Ingestion and Chunking

The KNOWX corpus contains nine documents collected from Microsoft, NVIDIA, and Infosys. The recorded ingestion output contains 6,256 chunks and approximately 5,000,829 extracted characters. The distribution is 893 chunks for Microsoft, 2,453 for NVIDIA, and 2,910 for Infosys. Let the processed corpus be

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\}, \tag{1}$$

where d_i is a text chunk and $N = 6256$. Each chunk retains metadata including organization and source document.

B. Semantic Retrieval

The all-MiniLM-L6-v2 model maps a query and document chunk into a 384-dimensional embedding space. Let e_q and e_d denote the query and chunk embeddings. Cosine similarity is

$$s_{\text{sem}}(q, d) = \frac{e_q^\top e_d}{\|e_q\|_2 \|e_d\|_2}. \tag{2}$$

Here, $e_q^\top e_d$ is the dot product and $\|\cdot\|_2$ is the Euclidean norm. KNOWX uses FAISS to search the semantic index. The precise FAISS index metric and normalization configuration should be checked against the implementation before publication.

C. BM25 Retrieval

BM25 ranks chunks using query-term statistics [2]:

$$s_{\text{BM25}}(d, q) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \tag{3}$$

Here, $f(t, d)$ is the frequency of term t in chunk d, $|d|$ is chunk length, avgdl is average chunk length, and k_1 and b are BM25 parameters. Exact parameter values were not supplied and must be verified before publication.

D. Reciprocal Rank Fusion

Let R_s and R_b be the semantic and BM25 ranked lists. A chunk's fused score is

$$s_{\text{RRF}}(d) = \frac{1}{k + \text{rank}_{R_s}(d)} + \frac{1}{k + \text{rank}_{R_b}(d)}, \tag{4}$$

where k is a rank-smoothing constant. If a chunk is absent from a list, that list contributes zero. More generally, for a set of rankings $\mathcal{R}$,

$$s_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}. \tag{5}$$

The supplied retrieval code uses $k = 60$. The fused candidates are passed to the reranker.

E. Cross-Encoder Reranking and Context Construction

The Cross-Encoder receives a query-passage pair and predicts a relevance score. Candidate passages are sorted in descending order of this score. KNOWX uses `cross-encoder/ms-marco-MiniLM-L-6-v2` and selects up to five passages for context construction. The selected passages are combined with source metadata and chunk text.

F. Answer Generation and Company Filtering

The query and constructed context are passed to a quantized Llama 3 8B Instruct model, executed locally using llama-cpp-python. The recorded configuration includes a context window of 4096 tokens, six CPU threads, and zero GPU layers. The system prompt instructs the model to use the supplied context and indicate when the context is insufficient.

The supplied implementation normalizes the selected company name and filters chunks by company metadata during semantic and BM25 retrieval. A further company check is applied after reranking and before source construction. This is retrieval-level filtering and is not, by itself, proof of complete production-grade tenant isolation or authorization security.

G. RRF Derivation

RRF uses the position of an item in each ranking rather than directly adding scores from different retrieval models. For a chunk d, define its contribution from ranking r as

$$c_r(d) = \frac{1}{k + \text{rank}_r(d)}. \tag{6}$$

Algorithm 1 KNOWX Hybrid Retrieval and Answer Generation

Require: Query q, selected company c, chunk collection $\mathcal{D}$
Ensure: Generated answer a and source list S

$R_s \leftarrow \text{SemanticSearch}(q, c)$
$R_b \leftarrow \text{BM25Search}(q, c)$
$R_f \leftarrow \text{RRF}(R_s, R_b)$
$C \leftarrow$ candidate chunk identifiers from R_f
$R \leftarrow \text{CrossEncoderRerank}(q, C)$
Remove candidates whose company metadata differs from c
$X \leftarrow \text{BuildContext}(R, \mathcal{D})$
if X is empty **then**
    $a \leftarrow$ insufficient-context response
else
    $a \leftarrow \text{LlamaGenerate}(q, X)$
end if
$S \leftarrow$ source metadata for retained candidates
return (a, S)

TABLE I
KNOWX CORPUS SUMMARY

Organization	Number of chunks
Microsoft	893
NVIDIA	2,453
Infosys	2,910
Total	6,256

The fused score is obtained by summing contributions:

$$s_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} c_r(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}. \tag{7}$$

Here, $c_r(d)$ is the contribution from one ranking, k is the smoothing constant, and $\text{rank}_r(d)$ is the one-based position of d in ranking r. Chunks with larger fused scores are placed earlier in the combined ranking.

Fig. 1 summarizes offline indexing and online question answering. The offline stage prepares semantic and keyword indexes. At query time, both retrieval methods produce candidates, rankings are fused, passages are reranked, and selected context is supplied to the local language model.

IV. Experimental Setup

A. Dataset

The corpus consists of nine enterprise documents from Microsoft, NVIDIA, and Infosys. The recorded ingestion output contains approximately 5,000,829 extracted characters and 6,256 chunks. Table I summarizes the chunk distribution.

B. Implementation Environment

The project was developed on Windows 11 with an AMD Ryzen 5 8640HS processor, 16 GB RAM, SSD storage, and integrated AMD Radeon graphics. The local Llama model was configured for CPU inference with six threads, a 4096-token context window, and zero GPU layers. The reported `hf-venv` environment used Python 3.13.2. Exact versions of all dependencies were not supplied.

C. Preliminary Retrieval Benchmark

The automated benchmark evaluated 30 questions, with ten questions for each organization. Questions were generated from selected source chunks, and the selected source chunk was treated as the one known relevant item for that query. Each method was assessed at a cutoff of five results. The methods compared were FAISS semantic retrieval, BM25 retrieval, Hybrid RRF, and Cross-Encoder reranking.

For one known relevant chunk, Precision@5 is the number of known relevant chunks in the top five divided by five. Recall@5 is the fraction of the known relevant set retrieved in the top five. The F1 measure is

$$F_1@5 = \frac{2P@5\,R@5}{P@5 + R@5}, \tag{8}$$

where $P@5$ and $R@5$ denote Precision@5 and Recall@5. Equation (8) is the harmonic mean of precision and recall.

Mean Reciprocal Rank is

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^{Q} \frac{1}{r_i}, \tag{9}$$

where Q is the number of queries and r_i is the rank of the known relevant chunk for query i. A zero contribution is assigned if it is not retrieved. nDCG@5 measures position-sensitive gain within the first five positions. Since only one known relevant chunk was labelled per query, these are known-source benchmark metrics, not full relevance judgments.

V. Results and Discussion

Table II reports the aggregate metrics from the supplied evaluation output. The Cross-Encoder achieved the highest reported values across the five metrics in this preliminary benchmark. BM25 outperformed FAISS on the reported aggregate measures, while Hybrid RRF did not exceed BM25 in this test.

The Cross-Encoder's MRR of 0.9833 indicates that the known relevant chunk was generally placed near the top of the returned ranking in this benchmark. Its nDCG@5 of 0.9877 likewise indicates strong ranking of the labelled source chunk under this relevance setup. These scores do not establish answer correctness, factual consistency, or performance on independently authored questions.

BM25 achieved a higher Recall@5 than FAISS in the supplied benchmark. One possible explanation is that the generated questions reused wording from their source chunks, which may favour lexical matching. This is an interpretation of the test construction, not a separately measured causal finding. Hybrid RRF did not outperform BM25 on the aggregate scores, showing that combining rankings does not guarantee improvement for every query set or evaluation design. The

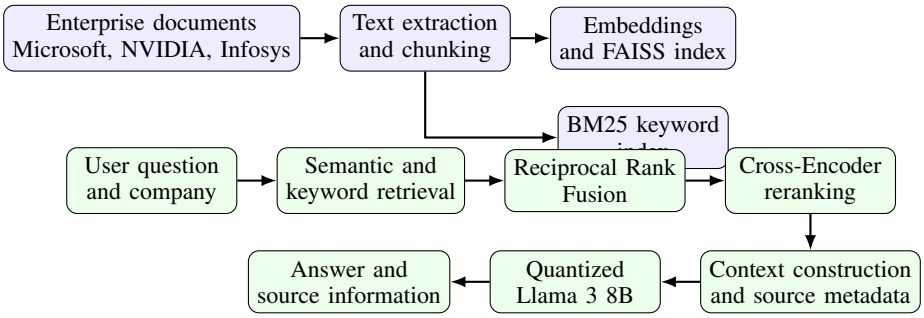


Fig. 1. KNOWX offline knowledge-base construction and online question-answering workflow.

TABLE II
PRELIMINARY KNOWN-SOURCE RETRIEVAL BENCHMARK RESULTS

Metric	FAISS	BM25	Hybrid RRF	Cross-Encoder
Precision@5	0.1333	0.1933	0.1800	**0.2000**
Recall@5	0.6667	0.9667	0.9000	**1.0000**
F1-score@5	0.2222	0.3222	0.3000	**0.3333**
MRR	0.4522	0.7500	0.6483	**0.9833**
nDCG@5	0.5053	0.8049	0.7122	**0.9877**

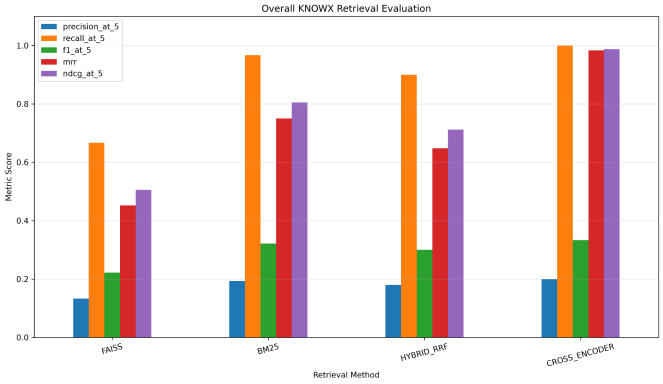


Fig. 2. Overall comparison of FAISS, BM25, Hybrid RRF, and Cross-Encoder retrieval using the five reported aggregate metrics.

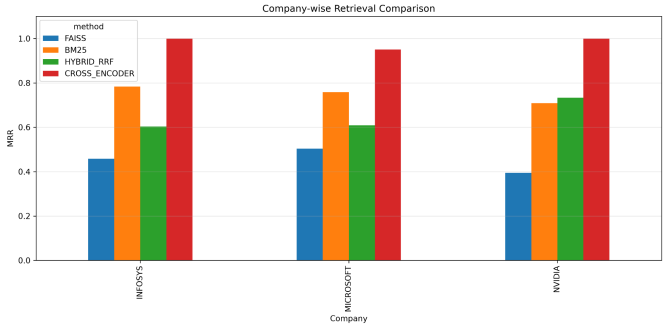


Fig. 3. Company-wise mean reciprocal rank comparison across Infosys, Microsoft, and NVIDIA, reproduced from the supplied plot. Exact underlying per-company values were not provided.

supplied company-wise MRR visualization shows the Cross-Encoder series as the highest plotted series for each of the three organizations. Because the chart was supplied only as an image and no per-company result file accompanied it, this observation is descriptive; no exact company-level MRR values or statistical comparisons are claimed.

A. Metric Visualization

The overall metric visualization in Fig. 2 reproduces the aggregate values supplied for FAISS, BM25, Hybrid RRF, and Cross-Encoder reranking. Fig. 3 reproduces the supplied company-wise MRR plot. The latter is retained as a visual comparison only: the underlying per-company numeric data were not supplied, so exact values are not transcribed or inferred from the raster bars. The aggregate table remains the authoritative source for the five overall metrics.

B. Limitations

The benchmark uses automatically generated questions and one known relevant source chunk per query. Other passages that may also be relevant were not labelled. Consequently, the reported precision and recall values are conditional on this incomplete relevance set. The benchmark is a preliminary known-source retrieval test rather than a comprehensive human-labelled retrieval evaluation.

No independently validated reference answers or human judgments were supplied for evaluating Llama 3 answer correctness, faithfulness, or completeness. Therefore, answer accuracy is not reported. A confusion matrix and ROC curve are not appropriate for the current ranked-retrieval and text-generation task because the system does not produce class labels or binary classifier predictions as its primary output.

A formal latency benchmark, repeated controlled experiments, statistical significance tests, ablation experiments, and comparisons with external language models were not supplied.

A supplied average-retrieval-time plot contains visible bars for FAISS, BM25, and Cross-Encoder, but the Hybrid RRF bar has zero height and the underlying timing records, run count, and measurement protocol were not supplied. It is therefore not treated as a verified latency benchmark, and no latency ranking or zero-second claim is made. The retrieval benchmark also does not establish production-grade security or complete tenant isolation. These aspects require separate evaluation. In particular, no ablation result is claimed for removing BM25, RRF, or Cross-Encoder reranking; such results should be added only after those configurations are run under the same benchmark.

VI. System Design Rationale and Requirements

A. Operational Requirements

KNOWX was designed around a constrained but practical use case: a user selects an organization, enters a natural-language question, and receives a response grounded in the indexed documents associated with that organization. This interaction imposes several requirements on the system. First, ingestion must preserve enough metadata to connect a retrieved passage to its original file. Second, retrieval must accommodate both lexical and semantic matches. Third, the answer generator must receive a bounded context rather than the entire corpus. Fourth, the system should run on the available development hardware without requiring a dedicated NVIDIA GPU. Finally, the interface should expose source information so that a user can inspect the evidence behind a generated response.

These requirements are implementation objectives, not claims that the current prototype satisfies every production criterion. In particular, the company selector and metadata filters provide organization-scoped retrieval in the current pipeline, but formal authorization, adversarial isolation tests, and multi-user security tests have not been reported. Likewise, displaying a source name supports traceability, but it does not by itself prove that every generated sentence is entailed by that source.

B. Functional and Non-Functional Requirements

The functional path begins with document ingestion and ends with an answer and source metadata. The ingestion component extracts text from the collected Word and PowerPoint materials, segments the text into chunks, and stores the resulting records. The indexing component creates dense representations for semantic search and a term-based index for BM25. The query component accepts a question and an organization selection, obtains candidate passages from both retrievers, fuses their rankings, reranks candidates, and constructs the context used by the generator.

The non-functional requirements include local execution, bounded memory use, reproducibility of indexing, and understandable source presentation. Local inference was selected because the available machine did not include an NVIDIA CUDA GPU. The model was therefore loaded through llama-cpp-python using a quantized GGUF artifact. This choice makes local execution possible in the recorded environment, but it does not imply that inference latency is suitable for every interactive workload. No controlled latency measurements are currently available.

C. Design Trade-Offs

A single retrieval method would simplify the architecture, but would also commit the system to one type of evidence matching. Dense retrieval can help when a question and a passage use different wording to express related ideas. BM25 can be useful when exact names, product terms, dates, or financial labels appear in both the query and the source. The hybrid design therefore preserves two retrieval channels before combining their ranked outputs.

RRF was selected as a rank-combination method because the component scores have different meanings and scales. Directly adding cosine similarity to BM25 scores would require a score calibration procedure. RRF instead uses rank positions. This avoids direct score-scale comparison, although the fusion result still depends on the candidate depth, the value of the smoothing constant, and the quality of each initial ranking. The implementation used $k = 60$ in the supplied retrieval code. The choice was not selected through a reported hyperparameter sweep and should be treated as an implementation setting rather than an empirically optimized value.

A Cross-Encoder adds another computation stage. Unlike a bi-encoder, which encodes a query and passage independently, a Cross-Encoder jointly evaluates the pair. This can provide a more query-specific relevance score but requires scoring each candidate pair. The system uses it after initial retrieval so that the more expensive model is applied to a smaller candidate set rather than to every chunk in the corpus. The current benchmark indicates strong known-source ranking under its automatic labelling scheme; it does not isolate the reranker's causal contribution under a human-labelled evaluation.

VII. Corpus Preparation and Data Representation

A. Source Collection

The corpus was assembled from documents associated with three organizations: Microsoft, NVIDIA, and Infosys. The selected materials include enterprise reports and related corporate documents. The project ingestion log records nine processed documents. It reports 5,000,829 extracted characters and 6,256 chunks after preprocessing. These counts describe the current local corpus snapshot. They should not be generalized to the full public document collections of the three organizations.

The collection was chosen to provide heterogeneous enterprise material rather than a single document type. Annual reports and business documents can contain narrative sections, tabular information, technical descriptions, and organization-specific terminology. This diversity creates practical retrieval challenges: a question may refer to a concept using a common phrase while the source uses a formal accounting or technical

term. A retrieval pipeline that combines lexical and semantic evidence is intended to address some of these differences.

B. Extraction and Normalization

Document extraction converts source files into text that can be indexed. In a production-grade document system, extraction quality should be checked for reading order, table structure, headers, footers, page boundaries, and non-text elements. The current project log confirms successful text extraction at the aggregate level, but it does not provide a per-document extraction audit or a measured extraction-error rate. Consequently, this paper reports the observed aggregate character and chunk counts but does not claim that all tables, charts, or layout relationships were preserved perfectly.

Normalization is necessary to make extracted content suitable for chunking and retrieval. Typical operations include whitespace normalization, removal of repeated empty lines, and preservation of meaningful punctuation and headings. Aggressive normalization can damage financial notation, identifiers, section numbers, and table content. The exact normalization rules and their effects should be documented from the source implementation before the paper is submitted as a reproducibility record. The checklist at the end identifies these details for verification.

C. Chunking as a Retrieval Unit

The system uses text chunks as the retrieval unit. Chunking converts a long document into smaller passages that can be embedded, indexed, ranked, and supplied to a language model. A chunk must be large enough to retain useful context but small enough to avoid diluting the representation with unrelated material. If chunks are too short, a definition and its qualification may be separated. If chunks are too long, the embedding may represent several topics at once and the language model may receive unnecessary text.

Let a source document D_j be segmented into an ordered sequence of chunks:

$$D_j \longrightarrow \{d_{j,1}, d_{j,2}, \ldots, d_{j,m_j}\}. \tag{10}$$

Here, D_j denotes source document j, $d_{j,i}$ denotes its ith chunk, and m_j is the number of chunks produced from that document. The full corpus is the union of the chunk sequences from all source documents. The project output records the final total of 6,256 chunks, but the precise chunk size, overlap, tokenizer, and boundary policy must be verified in the implementation. These values are therefore not inferred from the aggregate count.

D. Metadata and Traceability

A retrieved passage is more useful when it can be traced to its origin. KNOWX associates chunks with organization and source-document metadata. The retrieval pipeline uses organization metadata when filtering results. The backend also retains chunk identifiers and retrieval scores for internal source construction and evaluation. The intended user-facing presentation is narrower: it should show the source document name, rather than exposing implementation-level chunk identifiers and raw retrieval scores as if they were user-validated confidence values.

Metadata is important for debugging and auditing. If an answer appears incorrect, a developer can inspect the retrieved source record, compare the chunk text with the original document, and determine whether the issue arose during extraction, segmentation, retrieval, reranking, context construction, or generation. This layered diagnosis is preferable to treating the language model as the only possible source of error.

VIII. Retrieval Models and Mathematical Formulation

A. Dense Vector Representation

A sentence-embedding model maps a text sequence to a fixed-dimensional vector. In KNOWX, the selected model is all-MiniLM-L6-v2, whose model card specifies a 384-dimensional output representation. For a query q and a chunk d, the model produces vectors $e_q, e_d \in \mathbb{R}^{384}$. The cosine score in (2) measures the angular similarity between these vectors.

For clarity, cosine similarity can be derived from the normalized dot product. Define the normalized vectors:

$$\widehat{e}_q = \frac{e_q}{\|e_q\|_2}, \tag{11}$$

$$\widehat{e}_d = \frac{e_d}{\|e_d\|_2}. \tag{12}$$

Substituting these definitions into their dot product gives

$$\begin{aligned} \widehat{e}_q^{\top}\widehat{e}_d &= \left(\frac{e_q}{\|e_q\|_2}\right)^{\top}\left(\frac{e_d}{\|e_d\|_2}\right) \\ &= \frac{e_q^{\top} e_d}{\|e_q\|_2\|e_d\|_2} \qquad (13) \\ &= s_{\mathrm{sem}}(q,d). \qquad (14) \end{aligned}$$

The normalized dot product is therefore equivalent to cosine similarity. This equivalence is useful when an index implementation uses inner-product search over normalized vectors. The precise normalization and FAISS metric used by the current implementation must nevertheless be checked rather than assumed.

Dense retrieval has limitations. Embedding similarity is not a guarantee of factual relevance. A passage may be semantically close while referring to a different year, product, organization, or accounting measure. Similarity models can also underrepresent rare identifiers and exact strings. This is one reason KNOWX retains a lexical retrieval path.

B. BM25 Term-Based Ranking

BM25 is based on term statistics and document-length normalization. The inverse document frequency component gives greater weight to terms that are less common across the indexed collection. A common form is

$$\mathrm{IDF}(t) = \log\left(1 + \frac{n - N_t + 0.5}{N_t + 0.5}\right), \tag{15}$$

where n is the number of indexed chunks and N_t is the number of chunks containing term t. The exact IDF variant depends on the implementation. Equation (15) is a commonly used BM25 form and should be compared with the installed rank-bm25 implementation before being described as the exact formula used in the experiment.

BM25's term-frequency component saturates as a term occurs repeatedly in a chunk. Its length normalization adjusts for the fact that longer chunks have more opportunities to contain query terms. The parameters k_1 and b control term-frequency saturation and length normalization. Their actual values in the experiment are not available in the supplied index-construction record. The system's aggregate retrieval output does not expose these settings.

C. Rank Fusion Derivation

The semantic and lexical retrievers return ordered lists. Because their score scales are not directly interchangeable, the fusion step uses rank positions. For a set of retrieval systems $\mathcal{R}$, define the reciprocal-rank contribution of document chunk d from system r as in (6). Summing over all retrieval systems yields (7). In the two-retriever case used by KNOWX:

$$s_{\mathrm{RRF}}(d) = \frac{1}{k + \mathrm{rank}_{R_s}(d)} + \frac{1}{k + \mathrm{rank}_{R_b}(d)}. \tag{16}$$

Here, R_s is the semantic ranking, R_b is the BM25 ranking, and $k = 60$ in the supplied code. If a chunk is absent from one list, its contribution from that list is zero under the usual implementation convention. This convention and candidate depth should be confirmed against the exact code before final publication.

RRF rewards agreement across rankings. A chunk ranked highly by both systems receives two relatively large contributions. A chunk ranked highly by only one system may still appear in the fused list, but its score may be lower than a chunk supported by both. This property is useful when the retrieval systems have complementary strengths. It does not ensure that the fused list is better for every query; the benchmark results in Section V show that the measured hybrid aggregate did not exceed BM25 or Cross-Encoder reranking on the reported metrics.

D. Cross-Encoder Reranking

A Cross-Encoder takes the query and a candidate passage together as input and produces a relevance score:

$$s_{\mathrm{CE}}(q, d) = f_\theta([q; d]), \tag{17}$$

where f_θ denotes the trained Cross-Encoder, θ its parameters, and $[q; d]$ the model input formed from the query and passage. Unlike independent embedding generation, joint encoding allows token-level interaction between the query and candidate passage. The KNOWX implementation uses the model identifier `cross-encoder/ms-marco-MiniLM-L-6-v2`. The candidate set is reranked according to the model's scores.

Reranking is a second-stage process. It cannot recover a relevant chunk that was excluded from the candidate set before reranking. Therefore, the first-stage retrieval depth places an upper bound on the set of passages available to the Cross-Encoder. Candidate depth, maximum sequence length, truncation policy, and score ordering are implementation details that must be reported for reproducibility. Their values are not available in the supplied implementation record.

E. Context Construction

The selected passages are assembled into a context supplied to the generator. A context builder should balance evidence coverage against the language model's input budget. Repeated passages can waste tokens, while overly aggressive deduplication can remove complementary evidence. Ordering also matters: placing the most relevant passages earlier can make the intended evidence easier for the generator to use, although this must be evaluated rather than assumed.

Let the reranked candidate sequence be $R = (d_1, d_2, \ldots, d_m)$. A context selection function chooses a subset C subject to a token budget B:

$$C^\star = \arg\max_{C \subseteq R} \sum_{d \in C} s_{\mathrm{CE}}(q, d) \quad \text{subject to} \quad \sum_{d \in C} \ell(d) \le B. \tag{18}$$

Here, $\ell(d)$ is the token length of chunk d, B is the available context budget, and s_{CE} is the reranker score. Equation (18) expresses a general budgeted selection objective; it is not a claim that the current code solves this optimization problem exactly. The actual context assembly policy should be described from the implementation.

IX. Generation Layer and User Interaction

A. Local Language Model

The answer-generation component uses Meta Llama 3 8B Instruct in a quantized GGUF format. The local file is identified as `Meta-Llama-3-8B-Instruct-Q4_K_M.gguf`, with an approximate file size of 4.58 GB as recorded during setup. The runtime is llama-cpp-python. Hugging Face was used for authentication and access to model files; the model was not loaded through the Transformers inference API.

The recorded runtime configuration uses a context window of 4096 tokens, six CPU threads, and zero GPU layers. The development machine is a Windows 11 system with an AMD Ryzen 5 8640HS CPU, 16 GB RAM, SSD storage, and integrated AMD Radeon graphics. This configuration is reported to make the local model executable on the available system. No measured tokens-per-second, time-to-first-token, peak memory, or end-to-end latency values were supplied. The paper therefore does not make a speed or efficiency claim.

B. Grounded Answer Generation

The generation stage receives the user's question and the selected retrieved context. A grounded prompt should instruct the model to answer using the supplied evidence, avoid introducing unsupported facts, and state when the retrieved material is insufficient. The presence of retrieved context does not guarantee that the generated answer is correct. A generator can misread a passage, combine statements from different

periods, omit a qualification, or produce a plausible statement that is not supported by the evidence.

For enterprise questions, temporal and entity distinctions are particularly important. A financial value must be associated with the correct fiscal year, currency, and reporting basis. A technical statement must be associated with the correct product or system. A comparative question must not merge information from two organizations without preserving attribution. These risks motivate showing source document names and preserving source metadata in the backend.

C. Application Architecture

The application uses a FastAPI backend and a browser-based interface. The browser submits a question to the `/ask` endpoint. The backend calls the KNOWX pipeline and returns the generated answer and source information. The frontend renders the answer and source entries. The implementation files include `app/app.py`, `app/static/app.js`, `src/pipeline.py`, `src/hybrid_search.py`, and `src/reranker.py`.

This separation keeps retrieval and generation logic outside the browser interface. It also makes it possible to test the pipeline independently from the web application. However, a successful command-line pipeline test does not establish that all browser interactions work correctly. Browser-level validation should separately test request submission, error handling, organization selection, source rendering, and repeated queries. The recorded project evidence includes a successful pipeline test and backend startup logs; a complete automated browser test suite has not been supplied.

D. Error Handling and Abstention

A document question-answering application should not force a substantive answer when retrieval returns no useful evidence. An abstention response can indicate that the indexed documents do not contain enough information to answer the question. This behavior is especially important for questions about events, values, or policies outside the corpus.

A robust implementation should distinguish at least three conditions: no candidates were retrieved; candidates were retrieved but their relevance is low; and relevant passages were retrieved but the generator failed. The current project description includes an insufficient-context path in the pipeline algorithm. Thresholds, calibration, and error-category tests are not documented in the supplied implementation record. No measured abstention precision or recall is available.

X. Experimental Protocol and Evaluation Design

A. Evaluation Questions

The preliminary benchmark addresses a narrow question: how do the retrieval components rank a known source chunk for automatically generated questions derived from the corpus? It compares four stages or configurations: FAISS semantic retrieval, BM25 retrieval, Hybrid RRF, and Cross-Encoder reranking. It does not evaluate the factual correctness of generated answers.

A broader evaluation should answer separate questions. First, does hybrid retrieval improve recall over either individual retriever on independently judged queries? Second, does reranking improve the ordering of relevant passages? Third, do generated answers faithfully reflect the retrieved evidence? Fourth, does organization filtering prevent cross-organization evidence from entering the final context? Fifth, what are the latency and memory costs of the pipeline? These questions require different datasets and measurements; one aggregate retrieval table cannot answer all of them.

B. Query Set

The existing evaluation contains 30 questions, with ten associated with each organization. Questions were automatically generated and each question was associated with one known source chunk. The evaluation script then checked whether that chunk appeared in the retrieved results. This approach is useful as a smoke test for retrieval behavior and source traceability. It is not equivalent to a manually curated information-retrieval benchmark because the question wording, relevance labels, and alternative relevant passages were not independently judged.

The single-known-chunk setup can penalize a system for retrieving a different but equally relevant passage. It can also inflate the apparent completeness of the relevance set because relevant passages other than the designated source chunk are not labelled. For these reasons, Precision@5 and Recall@5 in this paper should be interpreted relative to the one labelled source chunk per question. They are not estimates of precision and recall over all relevant passages in the corpus.

C. Metric Definitions

Precision at rank cutoff K is the proportion of retrieved items in the top K that are relevant:

$$P@K = \frac{|\operatorname{TopK}(q) \cap G_q|}{K}, \tag{19}$$

where $\operatorname{TopK}(q)$ is the set of the first K retrieved chunks and G_q is the set of relevant chunks labelled for query q. Recall at K is

$$R@K = \frac{|\operatorname{TopK}(q) \cap G_q|}{|G_q|}. \tag{20}$$

Here, $|G_q|$ is the number of labelled relevant chunks. In the current automatic benchmark, $|G_q| = 1$ for each query. Therefore, a successful retrieval of the designated source chunk yields recall of one for that query, while failure yields zero.

The harmonic mean combines precision and recall:

$$F_1@K = \frac{2(P@K)(R@K)}{P@K + R@K}. \tag{21}$$

The measure is undefined when both precision and recall are zero unless a convention is adopted; evaluation code commonly records zero in that case. The aggregate values in this paper are copied from the evaluation output and should be interpreted according to the script's aggregation convention.

Mean Reciprocal Rank is defined in (9). For a query with the known relevant chunk at rank r_q, the reciprocal-rank contribution is $1/r_q$. If the chunk is not found within the evaluated result list, its contribution is zero. MRR averages this contribution over all Q queries.

Discounted cumulative gain at rank K is

$$\mathrm{DCG}\,@K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \tag{22}$$

where rel_i is the relevance grade at rank i. Ideal DCG, denoted $\mathrm{IDCG}\,@K$, is computed by sorting the known relevance grades in descending order. Normalized DCG is

$$\mathrm{nDCG}\,@K = \frac{\mathrm{DCG}\,@K}{\mathrm{IDCG}\,@K}. \tag{23}$$

For a binary, single-relevant-item setup, nDCG primarily reflects the rank of that one item. The exact relevance encoding and aggregation implementation should be checked against the evaluation script.

D. Reproducibility and Experimental Controls

A reproducible retrieval experiment should freeze the document corpus, chunking configuration, embedding model revision, index files, BM25 parameters, candidate depths, reranker revision, and query set. Each run should record software versions and the random seed for any component that uses randomness. If generated questions are produced by a model, the prompt and model version should also be retained.

The current output files are `evaluation_results/automatic_retrieval_20261030_205025.csv` and `evaluation_results/automatic_summary_20261030_205025.txt`. They provide aggregate retrieval metrics. The underlying per-query result file and exact question-generation procedure should be retained with the final submission. Without the per-query outputs, it is not possible to calculate confidence intervals or inspect which individual queries account for differences between methods.

E. Baselines and Ablation Scope

The current comparison includes four retrieval configurations. FAISS is the dense retrieval baseline; BM25 is the lexical baseline; Hybrid RRF combines the two; and Cross-Encoder is the reranked configuration reported by the evaluation script. The reported table should not be interpreted as a complete ablation study unless the evaluation code confirms that each configuration differs only by the component being studied and uses the same candidate pools and evaluation conditions.

A rigorous ablation would compare at least: dense retrieval alone; BM25 alone; dense plus BM25 without reranking; dense plus BM25 with RRF; and hybrid retrieval with Cross-Encoder reranking. Additional controlled experiments could vary candidate depth, RRF constant, embedding model, chunk size, overlap, and context budget. These experiments have not been supplied and are not fabricated in this paper. Their outcomes remain not available because controlled ablation runs were not supplied.

XI. Extended Results Interpretation

A. Aggregate Retrieval Results

The benchmark output reports Precision@5, Recall@5, F1@5, MRR, and nDCG@5 for each retrieval configuration. The Cross-Encoder row has the highest reported aggregate values across the five metrics. BM25 is second on MRR and nDCG@5, while Hybrid RRF is between BM25 and FAISS on the reported aggregate values. The exact measurements are shown in Table II; the interpretation is restricted to the benchmark's automatic known-source labels.

B. Semantic Retrieval

FAISS semantic retrieval reports Precision@5 of 0.1333, Recall@5 of 0.6667, F1@5 of 0.2222, MRR of 0.4522, and nDCG@5 of 0.5053. Under the benchmark's one-known-source-chunk convention, the result indicates that the designated chunk was retrieved within the first five results for a portion of the questions. The MRR and nDCG values also indicate that the position of the designated chunk varied across queries.

These values do not show that semantic retrieval is generally weak or ineffective. The query set is small and automatically generated, and the relevance set contains only one designated passage per question. A different query distribution or a broader set of relevance labels could change the relative ordering of methods. The result is best treated as a baseline measurement for this particular corpus snapshot and evaluation script.

C. Keyword Retrieval

BM25 reports Precision@5 of 0.1933, Recall@5 of 0.9667, F1@5 of 0.3222, MRR of 0.7500, and nDCG@5 of 0.8049. In this benchmark, BM25 retrieved the designated source chunk for nearly all questions within the top five. This result may be consistent with the query-generation procedure retaining important terms from the source chunk. It does not establish that BM25 will outperform semantic retrieval on independently authored questions or paraphrases.

The difference between BM25 and FAISS suggests that exact term overlap was useful for the current question set. A follow-up study should include questions written by people who have not seen the source passages, including paraphrases, abbreviations, multi-hop questions, and questions containing exact entity names. This would help determine whether the observed BM25 advantage is robust or partly a property of the automatic query construction process.

D. Hybrid Rank Fusion

Hybrid RRF reports Precision@5 of 0.1800, Recall@5 of 0.9000, F1@5 of 0.3000, MRR of 0.6483, and nDCG@5 of 0.7122. These values are above the FAISS-only results and below BM25 on the reported aggregate metrics. Thus, the current benchmark does not demonstrate that RRF improves over the BM25 baseline. It does show that the combined system retrieves the known source chunk for most queries under the benchmark's labels.

A possible explanation is that the BM25 ranking already performs strongly on the automatically generated questions, while adding semantic candidates changes the order of the fused list. This is a hypothesis, not a measured causal explanation. Query-level rankings and candidate overlap are needed to establish why the fused metrics are lower than BM25. A useful diagnostic would report how often the known source chunk appears in each individual candidate list, how its rank changes after fusion, and whether the added semantic candidates introduce useful alternative passages.

E. Cross-Encoder Reranking

The Cross-Encoder configuration reports Precision@5 of 0.2000, Recall@5 of 1.0000, F1@5 of 0.3333, MRR of 0.9833, and nDCG@5 of 0.9877. These are the highest values in the reported table. The high MRR and nDCG values indicate that the designated source chunk was generally placed near the top of the ranked list for the benchmark questions.

This finding must be interpreted with care. Reranking can only score candidates made available to it, and the evaluation setup uses one known relevant chunk per question. The aggregate metrics do not reveal whether the Cross-Encoder was reranking the same candidate pool as every other configuration, nor do they reveal how it behaves on queries outside the generated set. The result is encouraging evidence for the current test setup, but not proof of general answer quality or production readiness.

F. Comparison Across Metrics

Precision@5 and Recall@5 answer different questions. Precision@5 measures the fraction of the top five that are labelled relevant, whereas Recall@5 measures whether the labelled relevant set is retrieved. Because each query has one labelled relevant chunk, the reported recall is sensitive to whether that chunk appears anywhere in the top five. MRR and nDCG add rank sensitivity, rewarding a relevant item placed closer to the top.

The Cross-Encoder's reported MRR of 0.9833 and nDCG@5 of 0.9877 are close to one. These values indicate high ranking positions for the designated source chunk in this evaluation. They do not mean that 98.33 percent or 98.77 percent of generated answers are correct. The benchmark does not run answer generation, assess factual support, or use human answer ratings.

G. Statistical Uncertainty

The benchmark contains 30 questions. Aggregate point estimates alone do not show the uncertainty of the observed differences. A future analysis should retain per-query scores and compute paired confidence intervals, for example by bootstrapping queries. A paired significance test may be appropriate if its assumptions are checked and the test is specified before interpreting the outcome. No confidence intervals, p-values, or significance claims are reported here because the per-query data and a pre-specified statistical analysis were not supplied.

XII. Threats to Validity

A. Construct Validity

Construct validity concerns whether the chosen measurements represent the intended concept. The current benchmark measures retrieval of one designated source chunk. It does not fully measure retrieval relevance because additional relevant chunks are not labelled. It also does not measure answer correctness, completeness, citation faithfulness, or user satisfaction. Accordingly, the paper uses the term "known-source retrieval benchmark" rather than describing the results as overall KNOWX accuracy.

B. Internal Validity

Internal validity concerns whether observed differences can be attributed to the component being compared. The evaluation script should be inspected to verify candidate depth, filtering behavior, tie handling, missing-document treatment, and whether Cross-Encoder ranking operates on the same candidate set as the other configurations. If these conditions differ, the metric table compares pipeline configurations rather than isolating the causal effect of a single model component.

The automatic generation of questions is another possible source of bias. If the question generator preserves distinctive terms from the source chunk, lexical retrieval may benefit. If the questions are written to paraphrase the passage, semantic retrieval may benefit. The generation prompt and query examples should therefore be included in the supplementary material.

C. External Validity

External validity concerns generalization beyond the evaluated setting. The current corpus contains nine documents from three organizations. It is not a representative sample of all enterprise documents, languages, file formats, industries, or question types. The observed results should not be generalized to all enterprise knowledge-management systems. Larger collections and independently authored queries are needed to assess generalization.

D. Conclusion Validity

Conclusion validity concerns the strength of the inference drawn from the measurements. Thirty queries provide a preliminary signal but may not provide sufficient statistical power for small performance differences. The absence of per-query confidence intervals further limits inference. The paper therefore reports descriptive metrics without claiming statistical significance.

E. Implementation and Security Validity

The organization filter is implemented in the retrieval path, but that fact alone does not establish secure tenant isolation. A security evaluation would need to test unauthorized organization identifiers, malformed requests, metadata inconsistencies, cached results, direct API calls, and concurrent sessions. It would also need to examine file access controls and data retention. No such security audit has been supplied.

XIII. Practical Implications and Deployment Considerations

A. Source Transparency

Source transparency is a core requirement for enterprise question answering. A user should be able to identify which document supports a response and inspect the relevant original material when needed. KNOWX retains source document metadata in the backend and can return source names through the application interface. This provides a starting point for auditability, but a production interface should ideally include page or section references when those are available and reliable.

A source label should not be confused with a formal citation guarantee. A source may be relevant to the general topic but fail to support a particular sentence in the generated answer. Sentence-level attribution or evidence highlighting would provide stronger support, but it has not been implemented or evaluated in the supplied project evidence.

B. Local Inference

Running the language model locally can reduce dependence on a hosted inference service and can keep inference within the development machine. However, local execution also transfers responsibility for hardware capacity, model updates, access control, and operational monitoring to the deploying organization. Quantization reduces model storage and memory requirements relative to higher-precision weights, but may affect output quality. The present project has not measured the quality difference between quantization levels or compared local inference with a hosted model.

C. Resource Management

The recorded machine has 16 GB RAM and integrated graphics. The GGUF model is approximately 4.58 GB, and the runtime configuration allocates a 4096-token context window with six CPU threads. These configuration details help readers understand the execution environment. They do not establish the complete peak memory footprint, because runtime memory also depends on the context cache, candidate context, tokenization, and application overhead. Memory profiling remains not available because peak-memory measurements were not supplied.

D. Operational Monitoring

A deployed knowledge assistant should log enough information to diagnose failures without unnecessarily retaining sensitive document content. Useful operational measurements include request duration, retrieval duration, reranking duration, generation duration, error category, selected organization, and source identifiers. Logs should be access-controlled and retention should follow organizational policy. KNOWX is a project prototype; a complete operational monitoring and privacy review has not been reported.

XIV. Future Evaluation Plan

A. Human-Judged Retrieval Set

The next evaluation stage should create a query set written independently of the indexed source chunks. At least two assessors should judge candidate passages for each query using a documented relevance scale. Disagreements can be adjudicated or reported. The query set should include factual lookup, paraphrase, entity-specific, temporal, comparative, and multi-hop questions. It should also include unanswerable questions whose answers are absent from the corpus.

A multi-relevance test set would allow standard ranking metrics to reflect more than one relevant passage. It would also enable analysis of whether semantic retrieval contributes passages that BM25 misses, and vice versa. The test queries should be held out from any prompt or parameter tuning to reduce evaluation leakage.

B. Answer Quality Evaluation

Answer evaluation should be separate from retrieval evaluation. Human assessors can rate factual correctness, completeness, relevance, clarity, and faithfulness to the retrieved evidence. Each generated claim should be checked against its cited passage. A reference answer may be useful for some questions, but exact-match metrics alone are not sufficient for open-ended enterprise responses.

A useful evaluation protocol would report the number of questions, assessor instructions, rating scale, inter-rater agreement, adjudication process, and examples of unsupported or incomplete answers. Automated metrics may supplement human review but should not replace evidence inspection. All answer-quality metrics remain not available because no human-rated answer evaluation was supplied.

C. Latency and Resource Evaluation

End-to-end response time should be measured over a defined set of queries after a documented warm-up procedure. The report should separate embedding time, FAISS search time, BM25 search time, fusion time, Cross-Encoder time, context construction time, and generation time. Report median and tail latency, along with CPU and memory utilization. A single timing observation is not sufficient to characterize system performance.

D. Ablation and Sensitivity Analysis

Ablation experiments should test the contribution of each retrieval stage under identical query and relevance conditions. Candidate depth, chunk size, chunk overlap, RRF smoothing constant, and context budget should be varied systematically. The goal is not merely to find the highest score on the current 30 questions, but to determine whether the design choices remain useful on a held-out, human-judged query set.

E. Security and Privacy Evaluation

Before deployment with confidential enterprise documents, the system should undergo a security review. Tests should verify organization-level access control at the API boundary and retrieval layer, prevent cross-organization context leakage, and examine prompt injection embedded in retrieved documents. The application should treat document text as untrusted content rather than as instructions to the assistant. This is particularly important for RAG systems because retrieved content can contain text that attempts to influence model behavior.

XV. Reproducibility Notes

A. Software Components

The recorded implementation uses Python, Sentence Transformers, FAISS, rank-bm25, a Cross-Encoder from Sentence Transformers, llama-cpp-python, FastAPI, and Uvicorn. The embedding model is `all-MiniLM-L6-v2`; the reranker is `cross-encoder/ms-marco-MiniLM-L-6-v2`; and the generator is a quantized Llama 3 8B Instruct GGUF model. Exact package versions, model commit hashes, and index build parameters should be pinned in a machine-readable environment file before publication.

B. Artifacts

The implementation source code and reproduction materials are maintained separately from this paper.

C. Reproduction Procedure

A reproduction attempt should begin by obtaining the permitted source documents and confirming their versions. The environment should then be created using the pinned dependency specification. The ingestion stage should be run and its output counts compared with the reported nine documents, 5,000,829 extracted characters, and 6,256 chunks. The semantic and BM25 indexes should be rebuilt, followed by the retrieval evaluation script. Finally, the local GGUF model should be loaded and a small set of pipeline tests should be run.

Exact reproduction may not be possible if the original source files, model revision, or dependency versions are unavailable. The aggregate counts and benchmark metrics in this paper are therefore best understood as a record of the supplied project run, not a guarantee that a fresh environment will produce bit-identical output.

XVI. Additional Discussion

A. Why a Hybrid Architecture Is Reasonable

Enterprise queries often combine general concepts with precise identifiers. A question about a product family may require semantic matching, while a question about a named report section, fiscal year, or exact metric may require lexical matching. A hybrid architecture provides a way to combine these signals without assuming that one retriever is universally superior.

The benchmark results illustrate why hybrid retrieval should be evaluated rather than presumed to be better. In the current known-source benchmark, BM25 outperformed Hybrid RRF on the reported aggregate metrics. The Cross-Encoder configuration achieved the strongest ranking metrics. These results may motivate investigation of candidate generation and fusion settings, but they do not justify removing semantic retrieval without a broader query set. Semantic retrieval may be more valuable for paraphrased or concept-oriented questions that are underrepresented in the current benchmark.

B. Retrieval Is Not Generation

The pipeline contains two distinct quality problems. Retrieval determines whether the system obtains useful evidence. Generation determines whether the model produces an accurate, complete, and appropriately qualified answer from that evidence. A high retrieval score does not guarantee a correct answer, and a fluent answer does not demonstrate that the evidence was retrieved correctly.

For this reason, KNOWX should be evaluated as a sequence of components as well as an end-to-end application. Retrieval metrics assess ranking behavior. Answer metrics assess the response. Faithfulness review assesses whether the response is supported by the supplied passages. User studies assess whether the interface helps users complete real information tasks. The current evidence supports only preliminary retrieval evaluation and a recorded pipeline execution test.

C. Interpretation of the Current Prototype

The completed prototype demonstrates the integration of document ingestion, semantic indexing, lexical indexing, rank fusion, reranking, context construction, local language-model inference, and a FastAPI-based application path. The aggregate ingestion and evaluation outputs provide concrete evidence of a working project implementation. However, a prototype demonstration should not be described as a production deployment or a validated enterprise assistant.

The main technical result is the implementation and preliminary evaluation of a hybrid retrieval pipeline over a small multi-organization corpus. The strongest quantitative result is the Cross-Encoder configuration's known-source ranking performance on the automatically generated 30-question benchmark. The most important negative or qualified finding is that Hybrid RRF did not exceed BM25 on the reported aggregate metrics. Both observations are useful because they guide the next experiments instead of presenting the system as uniformly successful.

XVII. Conclusion

XVIII. Conclusion and Future Work

This paper presented KNOWX, a hybrid retrieval-augmented generation framework for enterprise knowledge management. The implementation processes documents from Microsoft, NVIDIA, and Infosys, producing nine processed

documents, approximately 5,000,829 extracted characters, and 6,256 chunks. It combines 384-dimensional sentence embeddings with FAISS semantic search, BM25 keyword retrieval, Reciprocal Rank Fusion, Cross-Encoder reranking, context construction, and local generation using quantized Llama 3 8B Instruct through llama-cpp-python. A FastAPI backend connects the pipeline to a browser-based interface.

The preliminary retrieval benchmark contains 30 automatically generated questions, ten per organization, and uses one known source chunk as the labelled relevant item for each question. Among the reported configurations, Cross-Encoder reranking achieved the highest aggregate values: Precision@5 of 0.2000, Recall@5 of 1.0000, F1@5 of 0.3333, MRR of 0.9833, and nDCG@5 of 0.9877. BM25 exceeded Hybrid RRF on the reported metrics. These observations are limited to the automatic known-source evaluation and should not be interpreted as generated-answer accuracy, complete retrieval relevance, or statistical evidence of general superiority.

The next stage should introduce independently authored queries and human relevance judgments, evaluate answer faithfulness and correctness, conduct controlled ablations, measure latency and memory, and test organization-level security. The implementation settings and document-processing parameters should also be recorded in a reproducible configuration. These steps are necessary before making claims about production readiness or general enterprise effectiveness.

XIX. Detailed Component-Level Test Plan

A. Document Ingestion Tests

The ingestion stage should be tested independently from retrieval. A useful test begins with a known source file and checks whether the expected text appears in the extracted output. The test should include ordinary paragraphs, headings, numbered lists, tables, footnotes, and repeated headers where those elements occur in the source collection. The expected output should be manually inspected for reading order and missing content. A document-level audit table should record the source filename, file type, extraction status, extracted character count, number of chunks, and any known extraction warnings.

The aggregate ingestion log in the current project reports nine documents and 6,256 chunks. That log is evidence that the ingestion pipeline completed for the recorded files. It is not a substitute for a document-by-document audit. In particular, the extracted-character count cannot reveal whether a table cell was placed in the correct row or whether a chart label was omitted. The authors should retain a sample of source pages and extracted text for manual verification.

B. Chunk Boundary Tests

Chunk boundaries can affect both retrieval and generation. A chunking test should inspect passages around section boundaries and verify that headings remain associated with the paragraphs they introduce. It should also check that important qualifications are not separated from the statements they qualify. For example, a financial paragraph may state a value and then explain whether it is annual, quarterly, reported, or adjusted. If these details are placed in different chunks, a retrieval system may return the value without its qualification.

A useful diagnostic is to sample chunks from each organization and record whether each chunk is coherent, self-contained, and attributable to its original section. The sample should include the shortest and longest chunks, chunks containing tables, and chunks near document boundaries. No chunk-coherence score has been measured for the current corpus. A formal chunk audit is therefore not available because no chunk-boundary audit was supplied.

C. Metadata Integrity Tests

Metadata integrity tests should verify that each chunk has a valid organization label and source-document identifier. They should also verify that the organization label agrees with the source file. A mismatch can cause an otherwise relevant passage to be excluded or, more seriously, returned under the wrong organization filter. Duplicate identifiers should be detected before indexing. A deterministic mapping from each chunk identifier to its source file should be retained.

For a multi-organization knowledge base, metadata integrity is not merely a display concern. It is part of the retrieval boundary. If metadata is incomplete or incorrect, organization filtering can behave incorrectly even when the filter logic itself is implemented as intended. The current project description confirms that organization metadata is used in filtering, but a complete integrity report has not been supplied.

D. Retrieval Unit Tests

Retrieval unit tests should use queries with known expected source passages. The tests should include exact phrase queries, paraphrases, named entities, numeric values, dates, abbreviations, and questions that cannot be answered from the corpus. Each test should record the expected organization, relevant document, relevant chunk identifiers, and acceptable alternative passages. The test should also verify that returned source metadata maps to the original file.

For semantic retrieval, unit tests should check that the query encoder and document encoder use the same model revision and preprocessing. For BM25, tests should check tokenization, punctuation handling, and the treatment of case and numerals. For hybrid retrieval, tests should check duplicate candidate handling and ranking ties. These tests can identify implementation defects that aggregate metrics may conceal.

E. Reranking Unit Tests

Reranking tests should confirm that the Cross-Encoder receives the intended query-passage pairs and that the output ordering is descending by relevance score. The test should include at least one query for which the known relevant passage is already near the top and one for which it begins lower in the candidate list. It should also verify that the reranker does not introduce passages that were not present in the candidate set.

The candidate pool should be saved for each test. This allows the team to distinguish a candidate-generation failure

from a reranking failure. If the relevant passage is absent before reranking, no reranker can recover it. If the relevant passage is present but moved downward, the issue is in scoring or ordering. The current aggregate benchmark does not provide enough detail to perform this diagnosis.

F. Generation Unit Tests

Generation tests should cover answerable and unanswerable questions, questions with conflicting values in different reporting periods, and questions that mention more than one organization. The expected behavior should be specified before running the model. For answerable questions, assessors should check whether the response preserves units, reporting periods, and qualifications. For unanswerable questions, the model should clearly state that the indexed material does not provide enough information.

Prompt-injection tests should include retrieved text that contains imperative language unrelated to the user's question. The generator should treat retrieved text as evidence, not as higher-priority instructions. This is a recommended test category for retrieval-augmented systems; no completed prompt-injection test results are claimed for the current prototype.

G. API and Interface Tests

The API should be tested with valid requests, missing fields, invalid organization names, empty questions, long questions, repeated requests, and model-loading failures. Each response should use a consistent schema. Error messages should be useful to the user without exposing file paths, tokens, or internal stack traces. The frontend should show a loading state while a request is running and should recover cleanly after a failed request.

Source rendering should be tested separately from answer generation. A response may contain valid source metadata even if the model fails to generate an answer. Conversely, an answer may be returned while source rendering fails. The application should make these states visible rather than silently presenting an incomplete result. A complete automated browser-test report is not available because no browser/API test report was supplied.

XX. Detailed Error Analysis Framework

A. Retrieval Misses

A retrieval miss occurs when the expected evidence is not present in the candidate set. Potential causes include extraction loss, an unsuitable chunk boundary, vocabulary mismatch, incorrect metadata, a query that is too broad, or insufficient candidate depth. These causes require different remedies. Increasing candidate depth may help when a relevant chunk is ranked just below the cutoff, but it will not repair missing source text or incorrect organization metadata.

A retrieval-miss analysis should store the query, expected document, expected chunk, candidate list, rank, and source metadata. Reviewers should inspect the original document and extracted chunk to determine whether the evidence was available to the retriever. Without this trace, it is difficult to distinguish model limitations from data preparation errors.

B. Ranking Errors

A ranking error occurs when a relevant passage is retrieved but placed below less relevant passages. BM25 ranking errors may be associated with term frequency, document length, or vocabulary overlap. Dense retrieval errors may be associated with embedding similarity or entity confusion. Fusion errors may occur when the combined ranking gives excessive weight to a list that is less useful for a particular query. Reranking errors may occur when the Cross-Encoder scores a passage highly because it shares topical words but does not answer the question.

An error taxonomy should label each query with one or more causes. The labels should be assigned after inspecting the query, retrieved passages, and original source. This would allow the team to identify whether the next improvement should target chunking, lexical processing, embeddings, fusion, or reranking.

C. Generation Errors

Generation errors include unsupported claims, incorrect numerical values, omitted caveats, temporal confusion, organization confusion, and failure to abstain. A useful audit should break an answer into atomic claims and check each claim against the retrieved context. A claim can be marked supported, partially supported, contradicted, or unsupported. This is more informative than assigning a single subjective score to an entire paragraph.

For financial and technical documents, numerical claims require special care. Reviewers should verify the value, unit, reporting period, and metric definition. A statement about revenue should not be confused with net income, operating income, or cash flow. A statement about a product's capability should not be generalized beyond the source's stated conditions. The current project has not supplied a claim-level answer audit.

D. Source Attribution Errors

Source attribution errors occur when the answer is linked to a document that does not support the relevant claim. This can happen even if the document is topically related. A source list should therefore be checked against the actual answer, not only against the query. If a generated answer combines facts from several documents, each fact should be traceable to the correct organization and reporting period.

A future version could present passage-level citations, section headings, or page references when extraction metadata supports them. Such functionality should be added only after verifying that the references point to the correct location in the original file. Incorrect page references can be more misleading than no page reference.

XXI. Alternative Design Choices

A. Dense-Only Retrieval

A dense-only design would embed each chunk and retrieve nearest neighbors for a query. Its principal advantage is a single retrieval index and a simple query path. It can be

suitable when questions are phrased differently from the source material. Its limitations include sensitivity to embedding quality, difficulty with exact identifiers, and the need to manage vector indexes as the corpus changes. In the current benchmark, FAISS-only retrieval produced the lowest reported aggregate values among the four compared configurations. This observation applies only to the current test set.

B. BM25-Only Retrieval

A BM25-only design is simpler than a hybrid architecture and can be effective for exact-term queries. It also offers an interpretable connection between query terms and retrieved passages. Its limitations include reduced ability to match paraphrases and concepts expressed with different vocabulary. The current benchmark shows stronger aggregate values for BM25 than for FAISS and Hybrid RRF. This supports retaining BM25 as a meaningful baseline and suggests that the query-generation process may have preserved useful lexical overlap.

C. Hybrid Retrieval Without Reranking

A hybrid system can combine dense and lexical rankings through RRF, weighted score fusion, or another rank aggregation method. RRF avoids directly combining incompatible raw score scales, but its outcome depends on the input rankings and rank constant. Weighted score fusion may allow greater control, but requires score normalization and parameter selection. The present project uses RRF with a constant of 60. The benchmark's Hybrid RRF values are lower than BM25's aggregate values, so the current results do not establish that fusion alone improves retrieval.

D. Hybrid Retrieval With Reranking

A reranker can refine a candidate list by scoring each query-passage pair. The benefit depends on candidate recall, model relevance, and the quality of the evaluation labels. In the current benchmark, the Cross-Encoder configuration reports the strongest aggregate ranking values. To attribute this improvement to reranking, the evaluation should ensure that the reranker receives a fixed candidate pool and that all configurations are compared under equivalent conditions. This experimental control should be documented before drawing a causal conclusion.

E. Alternative Generators

The current project uses a local quantized Llama 3 8B Instruct model. Alternative generators could be evaluated using the same retrieved context to isolate generation quality from retrieval quality. Such comparisons should hold the prompt, context, decoding settings, and question set constant. Evaluation should include factual correctness, evidence faithfulness, completeness, refusal behavior, latency, and resource use. No alternative-generator results are available for this paper.

XXII. Governance of the Evaluation Corpus

A. Document Provenance

A reproducible corpus should maintain a record of each source document's title, publisher, publication date, retrieval date, file format, and checksum. This record helps distinguish different versions of a report and prevents accidental mixing of documents from different fiscal years. A checksum can identify whether a file changed between indexing runs. The current aggregate ingestion output does not provide a complete provenance manifest, so the authors should create one before final archival.

B. Versioning and Re-Indexing

When a source document changes, the corresponding chunks and index entries should be updated. A robust process should remove obsolete chunks, insert new chunks, and rebuild or update the indexes in a controlled way. The system should also record the corpus version associated with each evaluation run. Otherwise, a metric change may reflect a different corpus rather than a change in the retrieval algorithm.

The current project stores processed documents and index artifacts locally. A future improvement is to add a corpus manifest and an index-build identifier. Each evaluation record could then reference the manifest version, model versions, and index build. This would make it easier to reproduce and compare results over time.

C. Licensing and Redistribution

Any shared implementation materials should clearly state which artifacts are included and which source materials users must obtain independently.

D. Privacy and Data Minimization

Even corporate reports can contain embedded metadata or material that should not be exposed in public artifacts. A production knowledge system should apply data minimization, access control, and retention policies. Logs should avoid storing full prompts and retrieved passages unless there is a clear operational need and an approved retention policy. The prototype's current local development configuration does not establish that these governance requirements have been implemented.

XXIII. Expanded Research Questions

The current implementation suggests several research questions that can be addressed in follow-up work. RQ1 asks whether hybrid retrieval improves retrieval quality over the strongest individual retriever when evaluated on independently written enterprise questions. RQ2 asks whether Cross-Encoder reranking improves ranking quality when all systems use the same candidate pool. RQ3 asks whether retrieval improvements lead to measurable gains in answer faithfulness and factual correctness. RQ4 asks how chunk size, overlap, and context budget affect retrieval and generation quality. RQ5 asks whether organization-level filtering can be validated under adversarial and concurrent access tests.

These questions should be investigated separately. RQ1 and RQ2 require a human-labelled retrieval test set. RQ3 requires answer-level assessment. RQ4 requires controlled parameter sweeps. RQ5 requires a security test plan. Combining all of these into one aggregate score would obscure the mechanisms that determine system behavior.

XXIV. Research Contributions Revisited

The principal contribution of KNOWX is an implemented end-to-end prototype that connects document processing, hybrid retrieval, reranking, local generation, and a web interface for a small enterprise-document collection. The system integrates established retrieval components rather than proposing a new embedding architecture or ranking algorithm. Its value is therefore in the engineering integration and the preliminary study of component behavior on the selected corpus.

The quantitative contribution is a transparent report of the available automatic retrieval benchmark. The results show that the Cross-Encoder configuration ranked the designated source chunk highly for the 30 generated questions, while BM25 outperformed Hybrid RRF on the reported aggregate metrics. Reporting this outcome, including the result that the hybrid method did not lead the comparison, provides a more accurate account than claiming that every additional component improved performance.

The work also identifies the next evidence required for a stronger research claim: independently authored questions, multi-passage relevance labels, human answer evaluation, controlled ablation, latency measurements, and security testing. The project should be considered a working prototype with preliminary retrieval evidence, not a validated production system.

XXV. Maintenance and Lifecycle of the Knowledge Base

A. Document Updates

Enterprise knowledge changes over time. A report may be replaced by a restated filing, a presentation may be revised, or a new annual report may introduce more recent values. A document assistant must therefore distinguish the date of a document from the date on which it was indexed. The retrieval result should retain enough provenance to identify the version used to generate an answer.

A document update workflow should first validate the new file, compute a checksum, and compare it with the previously indexed version. If the file is new, the ingestion pipeline can extract and chunk it. If the file replaces an older version, the system should remove or supersede the old chunk records before publishing the new index. The index build should be atomic from the application's perspective: a query should not observe a partially updated corpus in which only some chunks have been replaced.

The current KNOWX project records a static corpus snapshot of nine documents. An automated update scheduler, incremental indexing mechanism, or document-version management service has not been reported. These features are future work rather than current capabilities. A future experiment should measure the time required to ingest a new document and verify that old passages are no longer returned when a document is superseded.

B. Index Consistency

The semantic index, metadata file, and BM25 index must describe the same corpus version. If one artifact is rebuilt while another remains stale, candidate identifiers may point to mismatched metadata or different text. This can produce incorrect source labels and make retrieval results difficult to interpret. A corpus manifest should therefore record the number of chunks, their identifiers, source document names, and the build timestamp for each index artifact.

A simple consistency check can compare the set of chunk identifiers in the semantic metadata with the set of identifiers in the BM25 index. It should also verify that every identifier resolves to one processed chunk and one source document. The current aggregate ingestion logs establish the total chunk count but do not provide a published consistency audit. The audit result is not available because no index/metadata consistency audit was supplied.

C. Model and Dependency Updates

Changing the embedding model can invalidate the semantic index because the vector representation may change. A new embedding model should be applied to all chunks and queries consistently, and the index should be rebuilt. Similarly, changing the Cross-Encoder may change candidate ordering even when the candidate set is unchanged. A generator update can change answer style, factuality, and refusal behavior. Model versions should therefore be recorded as part of the experiment configuration.

Dependency updates can also change tokenization, ranking behavior, numerical precision, or model loading. The project should use a pinned requirements file or environment lock file. The exact versions of all libraries used in the recorded experiment are not available in the supplied evidence. The paper therefore names the principal libraries but does not invent version numbers.

XXVI. User-Centered Evaluation Considerations

A. Task Completion

A knowledge assistant should ultimately be evaluated by whether it helps users complete information tasks. A user study could ask participants to locate a specified fact in a set of enterprise documents using either manual search or KNOWX. The study should define task success before data collection, record completion time, and ask participants to rate confidence and usability. The sample size, participant characteristics, task design, and analysis plan should be documented.

No user study or task-completion measurement has been conducted for the current project. The system's conversational interface is an implementation feature, not evidence of improved productivity. Any claim that KNOWX saves time or improves decision-making must remain not established because no user study or productivity measurement was supplied.

B. Usability of Source Information

Source names help users identify which document contributed to an answer. However, a document title alone may be insufficient when a report contains hundreds of pages. A more useful interface could provide section titles, page numbers, or a short evidence excerpt. These additions should be implemented only if the extraction pipeline can preserve reliable location metadata.

The interface should also distinguish source evidence from generated explanation. For example, the answer may summarize several passages, while the source list identifies the documents from which those passages were retrieved. A user should not be led to believe that the source list independently verifies every sentence. Future interface testing should ask users whether they can locate and verify a claim from the displayed source information.

C. Handling Ambiguous Questions

Some questions are underspecified. A query such as "What was the revenue?" may not identify the organization, fiscal year, currency, or revenue definition. The system can either ask a clarifying question or return a qualified answer that explicitly states the scope inferred from the selected organization and retrieved passages. The preferred behavior should be tested with ambiguous and multi-interpretation queries.

The current project supports an organization selection in the retrieval path. It does not establish that all temporal or metric ambiguities are resolved automatically. A future evaluation should include questions that intentionally omit a year or use ambiguous financial terminology. Assessors can then determine whether the system asks for clarification, states its assumptions, or risks presenting a misleading answer.

XXVII. Model Risk and Evidence Discipline

A. Language Model Limitations

An instruction-tuned language model can produce fluent text that is not supported by the retrieved evidence. Quantization and local execution do not remove this risk. The generator may also fail to preserve a source's uncertainty, caveats, or scope. A retrieval-augmented architecture reduces dependence on model-internal knowledge by supplying external context, but the quality of the response still depends on retrieval, prompt design, model behavior, and context construction.

A safe system should not use confident wording when the context is incomplete. It should separate direct evidence from inference and avoid filling missing facts from general model knowledge when the task requires corpus-grounded answers. The application should make it easy to inspect the original source. These principles should be evaluated with adversarial and unanswerable questions rather than assumed from the use of RAG.

B. Numerical and Temporal Grounding

Enterprise reports frequently contain multiple periods and accounting measures. A retrieved passage can mention a quarterly result, while a nearby passage describes a full-year result. A generator that merges these statements can produce a plausible but incorrect comparison. A future answer-evaluation protocol should specifically test period alignment, currency, units, and the distinction between reported and adjusted measures.

One possible safeguard is to preserve headings and table labels during extraction and include them in the retrieved context. Another is to require the generator to state the reporting period and unit alongside each numerical answer. The current project has not reported a dedicated numerical-grounding test, so no quantitative reliability claim is made.

C. Prompt Injection in Retrieved Documents

Retrieved documents should be considered untrusted input. A malicious or accidental instruction embedded in a document could attempt to override the system's intended behavior. The generation prompt should clearly separate system instructions from retrieved evidence and state that document text is content to analyze, not instructions to follow. The pipeline should also test whether malicious text in a retrieved chunk can alter the answer or cause disclosure of unrelated context.

No prompt-injection robustness results are available for the current KNOWX prototype. This is a significant limitation for any future use with user-uploaded or externally sourced documents. A production system should include adversarial testing, input controls, output review, and access boundaries appropriate to the sensitivity of the documents.

XXVIII. Reporting Checklist for Future Experiments

A complete future experiment should state the research question, corpus version, query set, relevance-assessment procedure, retrieval candidate depth, model versions, runtime configuration, and aggregation method. It should publish the per-query results where permitted, not only aggregate averages. If the data cannot be redistributed, the authors should provide a clear description of how another researcher with lawful access can reproduce the evaluation.

The report should also distinguish exploratory experiments from confirmatory experiments. Exploratory runs can be used to identify useful parameter ranges, but final comparisons should use a held-out query set that was not used to select those parameters. If many configurations are tested, the authors should avoid reporting only the best-performing run without explaining the selection process.

For each reported metric, the paper should state the cutoff, relevance definition, averaging convention, and handling of missing results. For human evaluations, it should report the rating scale, number of assessors, assessor training, inter-rater agreement, and adjudication method. For latency tests, it should report hardware, warm-up, number of repetitions, and distribution statistics. These details allow readers to interpret results without assuming that different studies used identical procedures.

XXIX. Scope of the Present Study

The scope of this paper is deliberately limited to the implemented prototype and the evidence currently available. It documents the architecture and methods, reports the known corpus size and chunk distribution, and presents the aggregate retrieval benchmark. It does not claim a new ranking algorithm, a new embedding model, a production deployment, or a validated enterprise security architecture.

The benchmark is useful as an initial engineering evaluation. It shows that the system's retrieval components can be exercised over the selected corpus and that the designated source chunks are often retrieved within the top five for the generated questions. The Cross-Encoder configuration has the strongest reported rank-sensitive metrics, while BM25 is stronger than Hybrid RRF on the reported aggregate values. These observations identify concrete directions for a more rigorous evaluation.

A stronger research study would add human relevance labels, independently written queries, answer-level assessment, paired statistical analysis, ablations, and operational measurements. Until those experiments are completed, the current results should be treated as preliminary. This careful scope is important because a transparent statement of what has and has not been measured makes the implementation more reproducible and the research conclusions more credible.

XXX. Author Verification and Final Submission Notes

The authors should perform a final source-to-paper verification before submission. Every reported numerical value should be traceable to an ingestion log, evaluation output, or reproducible script. The corpus statistics in this paper are traceable to the recorded ingestion output, and the aggregate retrieval metrics are traceable to the automatic evaluation summary. Implementation details that were not available have been marked for confirmation rather than filled with assumed defaults.

The bibliography should be checked against the publisher or repository record for author order, title, year, venue, page range, and DOI. Software documentation references should include stable version or access information. The authors should also confirm that the institutional title page, author order, supervisor designation, and acknowledgments match the department's submission rules. The present IEEE conference layout is a research-paper format and may differ from a university's prescribed project-report template.

Finally, any source-code release should be reviewed for accidentally committed model weights, API tokens, private documents, temporary files, or environment folders. A clean release should include source code, setup instructions, permitted evaluation artifacts, and a clear statement about how users can obtain the source documents lawfully. The paper should be updated if any of these checks change the description of the implementation.

XXXI. Proposed Acceptance Criteria for the Next Prototype

The following acceptance criteria translate the limitations identified in this study into testable engineering outcomes. They are proposed criteria, not completed results. The project team should agree on thresholds before running the next evaluation so that the criteria are not selected after observing the test outcomes.

For ingestion, every indexed chunk should resolve to exactly one source document and organization, and the ingestion audit should record extraction failures rather than silently omitting them. For retrieval, every evaluation query should have a documented relevance set, and at least two assessors should review ambiguous cases. The held-out evaluation should include both answerable and unanswerable questions. Results should report Precision@K, Recall@K, MRR, and nDCG with a stated cutoff and aggregation convention.

For generation, every answer should be assessed for factual correctness and support by retrieved evidence. Numerical answers should preserve units and reporting periods. When the corpus does not contain the requested information, the system should abstain or ask for clarification rather than fabricate a response. The assessment should record unsupported claims separately from incomplete answers. A generated answer should not be counted as correct merely because it is fluent or includes a source title.

For organization filtering, tests should verify that a request scoped to one organization does not return chunks from another organization, including when the request is sent directly to the API. Tests should include malformed organization values, metadata mismatches, repeated requests, and concurrent sessions. These checks should be conducted with synthetic test records that do not expose confidential source material.

For operational performance, the next report should include median and 95th-percentile latency, peak memory use, and a breakdown of retrieval, reranking, and generation time. Each measurement should specify hardware, warm-up procedure, number of runs, and whether the model was already loaded. A performance threshold should be chosen based on the intended demonstration or deployment context rather than asserted without measurement.

For reproducibility, the corpus manifest, model identifiers, dependency versions, indexing parameters, evaluation queries, and per-query rankings should be versioned together. If a source document cannot be redistributed, its checksum and bibliographic metadata can still help identify the exact version used. These criteria provide a practical path from the current working prototype to a more rigorously evaluated research system.

Appendix A
Metric Calculation and Interpretation

The evaluation output reports aggregate metrics rounded to four decimal places. The table values should be preserved at the reported precision. Recomputing aggregate values from rounded per-query values can lead to small differences, so

the original unrounded evaluation output should be used if available. The formulas in Section IV define common ranking metrics; the exact implementation conventions must be verified against the evaluation script.

For the current benchmark, the single labelled chunk per question creates a special case. If the chunk is retrieved within the top five, recall for that query is one; otherwise, it is zero. Precision is one fifth when that single known relevant chunk is retrieved and zero when it is not, assuming the top-five list contains five unique items. This explains why the reported precision values are close to one fifth of recall values. The F1 values are consistent with this setting after aggregation, but the evaluation script should be consulted for exact macro- or micro-averaging behavior.

Appendix B
Implementation Verification Checklist

Before submission, the authors should confirm the exact chunk size and overlap; document extraction libraries; text normalization rules; BM25 k_1 and b; FAISS index type and distance metric; whether vectors are normalized; semantic and lexical candidate depths; RRF candidate handling and tie-breaking; Cross-Encoder maximum sequence length; context selection and truncation policy; generation prompt; model revision and quantization provenance; dependency versions; and the current frontend source-display behavior. These details affect reproducibility and should not be guessed from the aggregate logs.

Appendix C
Suggested Human Evaluation Form

A human evaluation should ask assessors to rate each answer on factual correctness, evidence faithfulness, completeness, and clarity. Each rating should be accompanied by a short justification and a reference to the supporting passage. Assessors should also be able to mark an answer as unanswerable from the corpus, partially supported, contradictory, or unsupported. The rating scale and adjudication procedure should be specified before evaluation begins. No ratings are reported in this paper because no human evaluation has yet been conducted.

Appendix D
Ethical and Responsible Use Considerations

Enterprise documents may contain confidential, proprietary, or personally identifiable information. Before indexing or publishing a document collection, project teams should verify that they have permission to process the files and should avoid including restricted content in public repositories or evaluation artifacts. Access to the application and stored indexes should be controlled according to the sensitivity of the source material.

Generated responses should be treated as decision-support outputs rather than authoritative records. Users should be able to inspect the source documents and verify important facts, especially financial, legal, operational, and compliance-related information. The model should be instructed to abstain when evidence is insufficient, and the system should be tested against misleading or malicious instructions embedded in retrieved documents. These are recommended safeguards; they are not claims that the current prototype has completed a formal compliance or security certification.

Acknowledgment

The authors thank their project guide for guidance and support during the development of KNOWX.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/1500000019.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[4] Sentence Transformers, *All-MiniLM-L6-v2 model card*, Accessed: 2026-10-30. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

[5] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759. DOI: 10.1145/1571941.1572114.

[6] R. Nogueira and K. Cho, *Passage re-ranking with BERT*, 2019. arXiv: 1901.04085. [Online]. Available: https://arxiv.org/abs/1901.04085.

[7] Sentence Transformers, *Retrieve and re-rank*, Accessed: 2026-10-30. [Online]. Available: https://sbert.net/examples/applications/retrieve_rerank/README.html.

[8] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. DOI: 10.1109/TBDATA.2019.2921572.

[9] Facebook Research, *Faiss: A library for efficient similarity search and clustering of dense vectors*, Accessed: 2026-10-30. [Online]. Available: https://github.com/facebookresearch/faiss.

[10] Meta AI, *Introducing meta Llama 3: The most capable openly available LLM to date*, Published: 2024-04-18, Apr. 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/.

[11] llama-cpp-python contributors, *Llama-cpp-python api reference*, Accessed: 2026-10-30. [Online]. Available: https://llama-cpp-python.readthedocs.io/en/latest/api-reference/.